

NEWVISION

IT Asset Management

Developer Handoff & Architecture

A complete technical briefing for a developer who has never seen this codebase.

Product	NewVisionITIS — multi-tenant IT asset inventory, helpdesk, chat, procurement
Audience	Incoming engineer with no access to the original authors
Codebase date	16 September 2026 (Prompts 0–38 + docs/ handover package)
Live demo	https://newvision-web.onrender.com API https://newvision-api.onrender.com
Repo	https://github.com/SatyamChouksey-88/newvision-it-admin
Classification	Internal. Contains architecture, env vars, and operational caveats — not production secrets.

This document was produced by reading the running codebase, Prisma schema, Nest modules, frontend routes, Docker/CI/Render config, and the in-repo decision log. Where the code and older markdown disagree, the code wins. Unclear items are labelled Unclear or Assumed.

How to read this: start with Overview and Architecture, then the flow that matches the ticket you were given. Local setup is section 7.

Table of Contents

Headings use Word Heading 1 / 2 / 3. Right-click the field below and choose Update Field if page numbers look stale after you edit the file.

1. Project Overview.....	5
1.1 What the system is.....	5
1.2 Who uses it	5
1.3 Why it exists.....	6
1.4 What it is deliberately not	6
1.5 Current shipping status.....	6
2. Architecture	7
2.1 High-level shape.....	7
2.2 Request pipeline	9
2.3 Multi-tenancy model	10
2.4 Frontend shell	11
2.5 Data model snapshot	11
3. Project Structure	12
3.1 Backend src layout	13
3.2 Frontend src layout.....	14
3.3 Docs you should not treat as current	14
4. Core Modules.....	15
4.1 Auth & users	15
4.2 Tenancy & platform	16
4.3 Assets (serialized hardware).....	16
4.4 Employees, locations, departments, categories.....	16
4.5 Accessories, consumables, issue kits	17
4.6 Asset requests.....	17
4.7 Maintenance	17
4.8 Support tickets (helpdesk)	17
4.9 Notifications & mail	18
4.10 Dashboard, search, reports.....	18
4.11 Import, saved views, reconciliation, webhooks, QR.....	18
4.12 Notes & manual correction.....	19

4.13 Procurement	19
4.14 Team Chat	19
4.15 Help	19
4.16 Frontend shared components worth knowing	19
5. Data & Process Flows	20
5.1 Login and session restore	20
5.2 Asset lifecycle (assign / transfer / retire / repair)	21
5.3 Maintenance / OEM repair	22
5.4 Asset / accessory request	23
5.5 Procurement	24
5.6 Support tickets and email-in	25
5.7 Import jobs and reconciliation	26
5.8 QR scan and scan-to-audit	27
5.9 Team Chat realtime	28
5.10 Signup (API) and first hour	28
5.11 Webhook delivery	28
5.12 Offboarding	29
6. Configuration & Environment	29
6.1 Backend variables	29
6.2 Frontend variables	31
6.3 Compose vs Render vs local .env	31
6.4 Secrets that must never be committed	31
7. Setup & Local Development	32
7.1 Docker Compose (recommended)	32
7.2 Local Node (no Docker for the app)	33
7.3 First-run empty estate	33
7.4 Common local pitfalls	33
8. Build, Test & Deployment	34
8.1 Build	34
8.2 Lint and typecheck	34
8.3 Backend unit tests	34
8.4 Backend API e2e	35
8.5 Playwright	35

- 8.6 CI 35
- 8.7 Production deploy (Render)..... 36
- 9. Conventions & Patterns 36
 - 9.1 API design..... 36
 - 9.2 Prisma / tenancy 37
 - 9.3 Frontend 37
 - 9.4 Naming..... 37
 - 9.5 Uploads 37
 - 9.6 Tests 37
 - 9.7 Design system 38
- 10. Known Issues / Technical Debt / Open Items 38
 - 10.1 Must-know product/ops gaps..... 38
 - 10.2 Test / process debt 39
 - 10.3 Stale documentation inside the repo 39
 - 10.4 Deliberate non-goals (do not file as bugs) 39
 - 10.5 Workarounds you will meet in the code..... 40
- 11. Glossary..... 40
- Appendix A — API surface cheat-sheet 41
- Appendix B — Related in-repo documents 42
 - How this handoff was produced 43

1. Project Overview

1.1 What the system is

NewVision (product UI title NewVisionITIS) is a multi-tenant SaaS web application for IT asset inventory in Indian companies. It answers the operational questions an internal IT team actually asks every day: how many devices exist, who has which laptop, where it sits, whether warranty is about to lapse, whether a repair is open, and whether a helpdesk ticket is overdue.

The product is a focused inventory + helpdesk + (on the Team plan) staff chat and procurement tool. It is not ServiceNow, not a CMDB, not an identity provider, and not a finance system. Those exclusions are deliberate and logged in FUTURE_IDEAS.md / DECISIONS.md — do not 'complete' them while fixing a ticket.

Technically it is two Node packages in one git repo: a NestJS 11 REST API with Prisma 7 on PostgreSQL 16, and a React 19 SPA built with Refine 5, Ant Design 5, and Vite 7. Realtime staff chat uses Socket.IO on the /chat namespace. Production is a Render Blueprint (Singapore region, demo only; India hosting on request).

1.2 Who uses it

Five roles share one SPA. The sidebar, home screen, and API all change with the role. The API is authoritative; hiding a button is not security.

Role	Typical person	What they see / can do
SUPER_ADMIN	IT head / workspace owner	Everything, including users, asset hard-delete, workspace export/delete, audit log. Production MFA (TOTP) is implemented but currently forced off on Render so the demo Super Admin can sign in.
IT_ADMIN	Day-to-day IT estate owner	Full lifecycle, org CRUD, import/export, procurement, webhooks, settings. Cannot manage users or hard-delete assets.
IT_SUPPORT	Helpdesk / floor walker	Read estate, manage maintenance + tickets, run reports, staff chat, fulfill requests, scan-to-audit. No procurement nav (except vendor name picker for repairs). Account via avatar menu.
MANAGER	Line manager	Team home, team devices/people, approve direct-report requests, team tickets, team requisitions, reports labelled "your team".
EMPLOYEE	Anyone with kit	My IT home, My kit, raise a request, own tickets, Help, phone-width bottom nav. No chat, no procurement, no admin grids.

There is also a public, unauthenticated audience: anyone who photographs a QR sticker. That page is intentionally PII-safe (no assignee name, no serial, no cost, no history).

1.3 Why it exists

The original internal brief was a company with roughly 1,250 serialized assets and 1,180 employees across three offices (Pune, Hyderabad, Bhopal) tracking kit in spreadsheets. NewVision replaces that with one system that enforces a real asset lifecycle, writes an append-only audit trail, and gives each role a shell that does not look like every other role.

Prompt 37 productized the same codebase as a sellable multi-tenant SaaS: self-serve signup, a 14-day Team trial that falls back to Starter (no vendors/chat/maintenance), shared Postgres with tenantId isolation, and in-app hooks for export / delete / billing. Legal entity, GST invoicing (Zoho Books), DPA/MSA PDFs, and certifications stay outside this repo.

1.4 What it is deliberately not

- Not a CMDB, service catalog, change/release manager, or SLA rules engine (first-response SLA is three numbers in Settings).
- Not live AD/Entra/Intune/MDM. Joiner-mover-leaver is CSV import + reconciliation + offboard endpoint.
- Not a workflow designer. Asset requests are one manager step. Requisition approval is a To/Cc matrix, default parallel.
- Not a money-moving system. Procurement tracks vendors, POs, GRNs, invoice match status, and contracts. It does not pay anyone.
- Not a perfect Teams clone. No calls, meetings, guests, GIFs, or federation.
- Not dark mode. Light tokens only; OS dark preference is forced back to light.
- Not a phone-width rewrite of the IT Admin Excel grids. nv-phone (≤639px) is Employee + lookup; Chat/procurement show a desktop-only banner.
- Not Redis/Bull. Import jobs run in-process via setImmediate; files live as bytea on the job row.

1.5 Current shipping status

Phases 0–4 and Prompts 2–38 described in PROGRESS.md are implemented in this tree. Production URLs exist on Render. Two operational proofs are still open and must not be claimed as done: a real Resend inbox send, and a real IMAP ingest. Those need operator secrets on the Render dashboard.

Surface	Status
Core inventory (assets, people, locations, assign/transfer/retire)	Shipped, API-enforced lifecycle
Maintenance / OEM claim fields	Shipped
Helpdesk + email-in code path	Shipped; live mailbox unproven
Accessories / consumables / issue kits / My kit	Shipped (Prompt 38)

Procurement	Shipped (Prompt 23 + 36)
Team Chat	Shipped (Prompts 24/34); live two-session verified locally
Multi-tenant SaaS + signup API	Shipped (Prompt 37); frontend /signup and /trust pages exist but are not registered in App.tsx — see §10
Security hardening	Shipped (Prompt 32); rotate live secrets still an ops task

2. Architecture

2.1 High-level shape

Three runtime pieces: a static (or Vite-dev) SPA, a long-lived NestJS process, and one PostgreSQL database. The API is a persistent web service on purpose — Socket.IO, @nestjs/schedule crons, and the IMAP poller cannot live on a serverless function. That is why the project stayed on Render rather than moving to a lambda host (DECISIONS.md, Prompt 27).

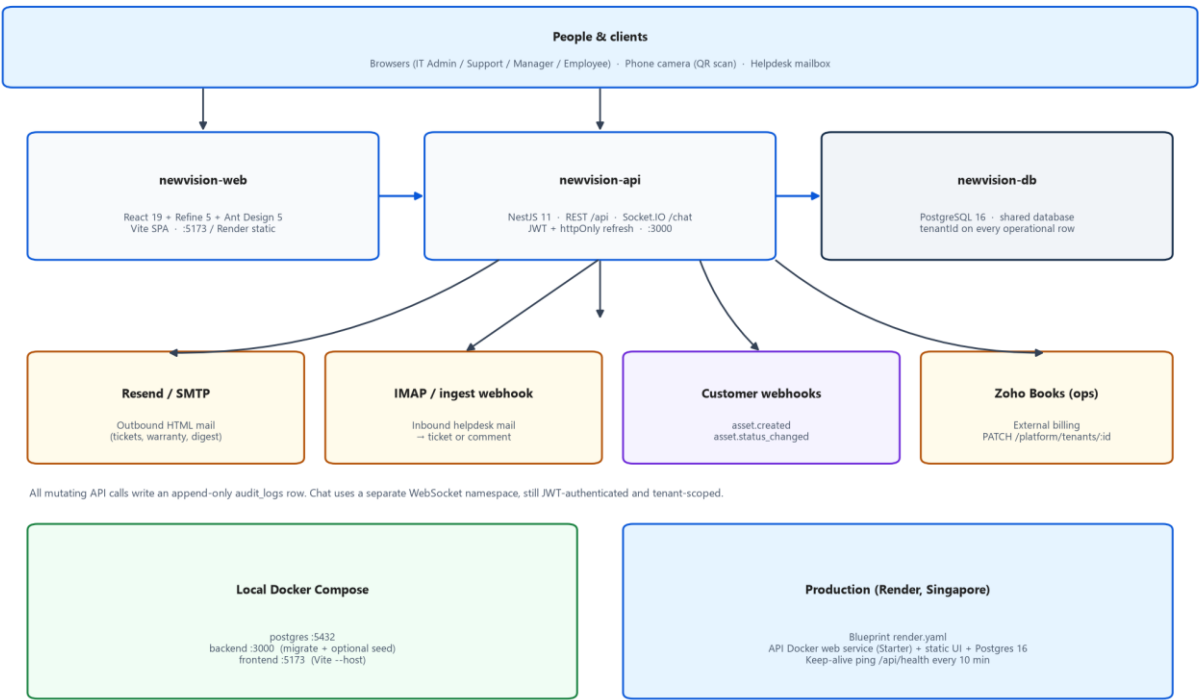


Figure 1. High-level system architecture: clients, SPA, API, Postgres, and external services.

Why each major technology is in the stack:

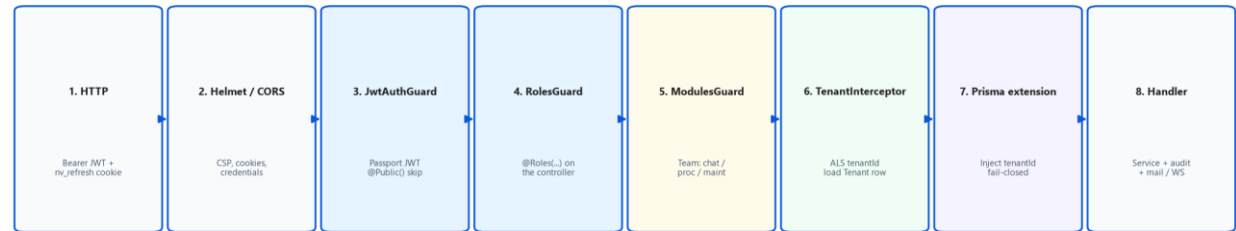
Technology	Where	Why it is here (or the constraint)
------------	-------	------------------------------------

TypeScript 7	Both packages	Nest 11 schematics wanted TS ≥ 6 ; current stable was 7. Installs use legacy-peer-deps.
NestJS 11 (CommonJS)	backend/	Nest 12 is ESM; Jest cannot require those packages. v11 keeps unit/e2e working.
Prisma 7.10.0 + @prisma/adapter-pg	backend/	Pinned off the CLI RC. Prisma 7 removed datasource URL from schema.prisma; connection lives in prisma.config.ts; PrismaClient needs a driver adapter. pg.Pool max: 10.
PostgreSQL 16	compose + Render	Single shared DB; tenantId column isolation, not database-per-tenant (Render Blueprint cannot provision a DB per signup).
React 19 + Refine 5 + Ant Design 5	frontend/	@refinedev/antd requires AntD 5, not 6. Refine gives resources/auth/dataProvider; custom dataProvider maps _start/_end/_sort/_order.
Vite 7	frontend/	SPA build. Production chunk split: charts, antd (~1.2 MB irreducible), refine, react-vendor.
JWT + httpOnly cookie	auth	Access token 30m (compose/Render) in sessionStorage. Refresh 7d in nv_refresh cookie. Residual XSS on the access token is documented, not unsolved.
Socket.IO	chat	Staff chat live events. JWT in handshake auth.token.
Resend then SMTP then console	mailer	Render Free blocks outbound 587/465. HTTPS Resend is the production path.
Biome 2.5	both	typescript-eslint refuses TypeScript 7. Biome lints and formats; unsafeParameterDecoratorsEnabled for Nest.
Jest 30 + @swc/jest	backend tests	ts-jest/ts-node break on TS 7. Specs import @jest/globals explicitly.
Playwright + axe-core	frontend/e2e	Real UI against a seeded API on :3000. Accessibility is a failing test, not a separate script.
Docker Compose	local	Postgres + API + Vite. Frontend image is the dev server, not the production static build.

Render Blueprint	prod	render.yaml names newvision-db / newvision-api / newvision-web. Region singapore.
------------------	------	---

2.2 Request pipeline

Every HTTP request that is not `@Public()` walks the same global stack, registered in `app.module.ts`. Order matters: authenticate, then roles, then plan modules, then tenant ALS.



Global models Role, Permission, Tenant skip tenant injection.

Login / signup / health / public scan / QR use `@Public()` and often `runUnscoped()`. Missing ALS on a tenanted model throws.

Figure 2. Authenticated request pipeline from HTTP to Prisma.

Key files:

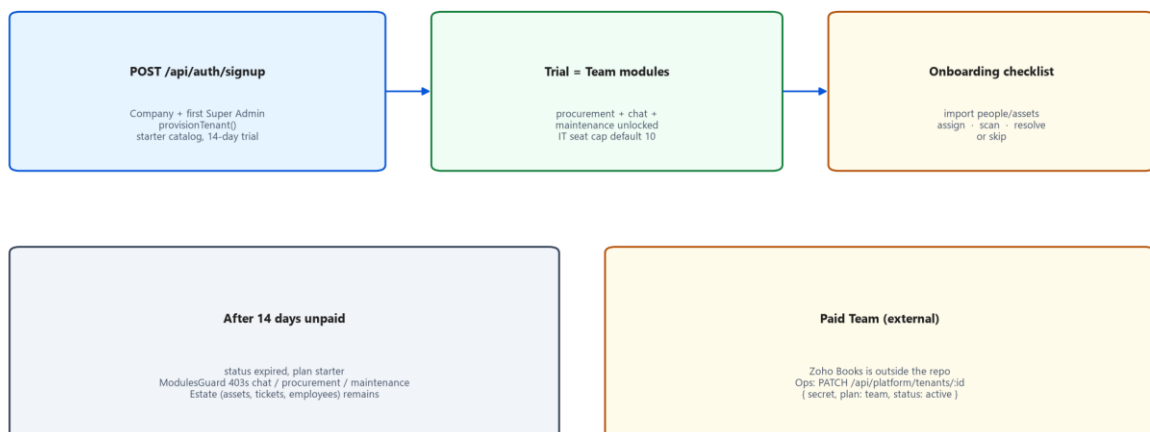
- [backend/src/configure-app.ts](#) — Helmet CSP (default-src none), cookie-parser, CORS with credentials, ValidationPipe whitelist, global prefix api, Socket.IO adapter, trust proxy 1.
- [backend/src/common/guards/jwt-auth.guard.ts](#) — skips WebSocket (the gateway authenticates itself) and `@Public()` routes.
- [backend/src/common/guards/roles.guard.ts](#) — reads `@Roles()` metadata; empty list means any authenticated user.
- [backend/src/tenancy/modules.guard.ts](#) — 403s `/api/vendors`, `/api/purchase-*`, `/api/procurement`, `/api/chat`, `/api/maintenance` when the tenant's effective modules disallow them. Trial tenants must finish or skip onboarding before procurement/chat.
- [backend/src/tenancy/tenant.interceptor.ts](#) — `runWithTenant(user.tenantId)`; loads Tenant; auto-expires a lapsed trial to starter.
- [backend/src/tenancy/prisma-extension.ts](#) — injects `tenantId` on reads/writes. Role, Permission, Tenant are global. Missing ALS fail-closes except `findUnique` (login race).

- `backend/src/tenancy/context.ts` — `AsyncLocalStorage`. Always enter via an async callback so Prisma's microtask still sees the store.

```
// backend/src/app.module.ts (providers)
{ provide: APP_GUARD, useClass: JwtAuthGuard },
{ provide: APP_GUARD, useClass: RolesGuard },
{ provide: APP_GUARD, useClass: ModulesGuard },
{ provide: APP_INTERCEPTOR, useClass: TenantInterceptor },
```

2.3 Multi-tenancy model

Prompt 37 chose shared Postgres + `tenantId` on every operational table (option B), not database-per-tenant. Self-serve signup cannot provision a new Render database. Roles and Permissions stay global. User.email stays globally unique (one login per human). Business keys such as `location.code`, `employeeCode`, `assetCode`, ticket numbers, and vendor codes are unique per tenant: `@@unique([tenantId, field])`.



Isolation: one Postgres, `tenantId` on every operational table. User.email is globally unique. Business keys are `@@unique([tenantId, field])`. Prisma `$extends` injects `tenantId`; missing context fail-closes.

Legal/GST/DPA/MSA PDFs are not in this repo. In-app hooks: `/trust (placeholder)`, `GET /tenant/export`, `DELETE /tenant`, `GET /tenant/billing`.

Figure 10. Signup, 14-day Team trial, onboarding lock, and Starter fallback.

Effective modules (`backend/src/tenancy/plans.ts`):

- Trial that has not expired → Team modules (procurement, chat, maintenance) regardless of the JSON column.
- Team + active → the JSON modules column (usually all true).
- Expired trial, cancelled, or Starter → `STARTER_MODULES` (all three false). Assets, employees, tickets, scan, settings remain.

- Assets are not metered. The only cap is IT seats (Super Admin + IT Admin + IT Support), default 10.
- Onboarding JSON with skipped: true (the demo seed) does not lock modules. Null onboarding (legacy tenant 1) is treated as complete.

Payment is external. After Zoho Books records a Team subscription, ops call PATCH /api/platform/tenants/:id with PLATFORM_ADMIN_SECRET. That endpoint is @Public() and returns { ok: false } on a bad secret rather than 401, to avoid leaking that the hook exists.

2.4 Frontend shell

frontend/src/App.tsx is the routing table. Refine wraps Ant Design ConfigProvider (light algorithm only). Most authenticated routes sit inside ThemedLayout with a custom AppSider (216px) and Header (52px). Two exceptions deliberately leave that shell so they do not grow a second left rail: /help/* and /chat.

- **Auth:** authProvider talks to /auth/login. Access token + user JSON go to sessionStorage keys newvision:token / newvision:user. Axios sends Bearer + withCredentials: true so the refresh cookie travels.
- **401 handling:** [providers/axios.ts](#) single-flights POST /auth/refresh and retries once. Failure clears session and sends the user to /login?to=...
- **RBAC in the UI:** [access.ts](#) mirrors backend ROLE_PERMISSIONS. navForRole builds the sider. RoleRouteGuard blocks URL guessing. accessControlProvider hides Refine resources.
- **Data:** [providers/dataProvider.ts](#) maps Refine pagination to Nest's _start/_end/_sort/_order and reads { data, total }.
- **Lazy routes:** every page is React.lazy. An inner Suspense around Outlet is required — React Router does not bubble Outlet suspends to a Suspense around Routes.
- **Tablet:** below 992px Refine's hamburger drawer; 992–1023px TabletCollapse icon-collapses the sider.
- **Phone:** nv-phone ≤639px. EmployeeBottomNav. IT lists become cards. Chat/procurement DesktopOnlyBanner.

2.5 Data model snapshot

The full schema is backend/prisma/schema.prisma (~2,100 lines, 60+ models). Postgres table names are snake_case via @map. Every operational model includes tenantId with a relation back to Tenant. The diagram below is the spine a new developer should memorize; procurement and chat expand from Tenant the same way.

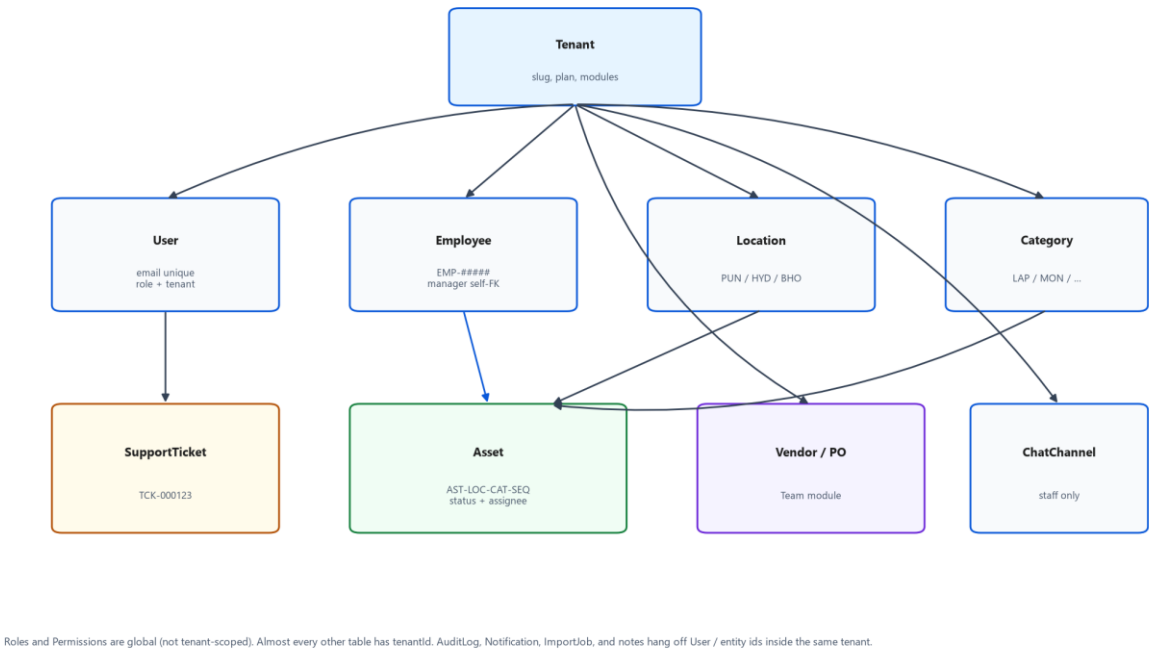


Figure 15. Core data relationships (simplified).

Asset codes default to AST-{locationCode}-{categoryCode}-{sequence} (backend/src/assets/asset-code.ts). Create also accepts a custom code (letters/digits/hyphen, min 3, unique per tenant). Renaming an asset invalidates old QR stickers — the UI confirms.

Ticket numbers are assigned after insert: a temporary TMP-... is replaced with TCK-{id padded to 6}.

3. Project Structure

This is a monorepo with two independent package.json files. There is no npm workspaces hoisting. Root holds compose, Render, CI, and the product decision docs.

IT_ADMIN/	
├─ backend/	NestJS API + Prisma
│ └─ prisma/	schema.prisma, migrations/, seed.ts
│ └─ src/	one folder per Nest module
│ └─ test/	Jest API e2e (real Postgres newvision_test)
│ └─ Dockerfile	multi-stage; ships src so seed can import RBAC
│ └─ docker-entrypoint.sh	migrate deploy → optional seed → node
├─ frontend/	Refine + AntD Vite SPA
│ └─ src/pages/	one folder per screen area
│ └─ src/components/	shared chrome (DataGrid, Header, AppSider, ...)
│ └─ src/providers/	auth, axios, session, dataProvider
│ └─ src/help/	in-app docs model (articles.ts)
│ └─ e2e/	Playwright + fixtures/
└─ public/brand/	logos, favicon

		public/docs/screenshots/	Help PNGs
		Dockerfile	Vite dev server (compose only)
		docker-compose.yml	postgres + backend + frontend
		render.yaml	Render Blueprint
		.github/workflows/	ci.yml + keep-alive.yml
		design-reference/	visual source of truth HTML + DESIGN_TOKENS.md
		docs/	BACKUP.md + this handoff

3.1 Backend src layout

Folder	Responsibility
auth/	Login, refresh cookie, signup, password reset, TOTP, rate-limit
tenancy/	ALS context, Prisma extension, plans, provision, sample-company, guards, tenant HTTP
prisma/	PrismaService with pg Pool + tenant extension Proxy
common/	RBAC, guards, decorators, lifecycle/warranty/uploads/password/scan-url/hosting
assets/	CRUD, assign/transfer/retire/bulk, QR, audit-by-code, lifecycle.ts, asset-code.ts
employees/	CRUD, offboard, history, checklists, self-edit /me
locations/ departments/ categories/	Org structure CRUD
accessories/ consumables/	Quantity stock, checkout/issue, low-stock
asset-requests/	pending → approved rejected → fulfilled
maintenance/	Repair tickets coupled to asset status
tickets/	Helpdesk, digest, SLA, email-inbox, templates
chat/	HTTP + ChatGateway + presence/realtime
procurement/	Vendors, PRs, POs, GRNs, invoices, contracts
dashboard/ search/ reports/	Home KPIs, global search, CSV/PDF
import-export/ import-jobs/ saved-views/ reconciliation/	Phase 3 governance
notifications/	In-app + MailerService + warranty cron + HTML templates
audit/ notes/ records/	Audit reader, append-only notes, manual override
webhooks/ qr/ public-assets/	Outbound HMAC webhooks; QR PNG; unauthenticated scan card
users/ health/	User admin; GET /api/health

3.2 Frontend src layout

Path	Responsibility
App.tsx	Refine resources + React Router table
access.ts / chrome.ts / theme.ts / types.ts	RBAC nav, shell helpers, AntD tokens, shared TS types
pages/dashboard.tsx	Role-specific homes (IT KPIs vs IT Support My work vs Manager Team vs Employee My IT)
pages/assets employees locations ...	One resource per folder; list/show/create/edit
pages/help/	MkDocs-Material-style in-app docs
pages/chat/	Full-page Teams-style chat (own chrome)
pages/login.tsx signup.tsx trust.tsx	Public marketing/auth screens – see routing caveat in §10
pages/scan.tsx	Public QR card + staff audit form
components/DataGrid/	Excel-grade table (sort/filter/resize/reorder/density/CSV)
components/AppSider.tsx Header.tsx	Custom shell; not Refine ThemedSider
providers/	Axios client, sessionStorage, Refine auth + data + accessControl
help/articles.ts	Structured help content; search/ToC are generic over this model

3.3 Docs you should not treat as current

The repo has many markdown status files written at different prompts. Several contradict the code (Nest 12 vs 11, AntD 6 vs 5, JWT 8h vs 30m, “no refresh tokens”, “no public signup”). Use this handoff plus DECISIONS.md (newest notes at the bottom of each phase) plus PROGRESS.md. PROJECT_DOCUMENTATION.md says itself that sections 1–11 stop at Prompt 19.

4. Core Modules



Figure 3. Backend NestJS modules grouped by concern.

Each subsection names the Nest module, the primary HTTP prefix, the frontend route, and the rule you will break if you “simplify” it.

4.1 Auth & users

[backend/src/auth/](#) · [backend/src/users/](#) · [frontend/src/providers/authProvider.ts](#)

POST /api/auth/login returns { access_token, user } in production (refresh is cookie-only). Local/dev also echoes refresh_token in the JSON so tests stay simple (includeRefreshInBody). Passwords are bcrypt. Unknown emails still run bcrypt against a dummy hash so timing does not leak existence. Login DTO MinLength is 1 — we do not tell attackers “password too short”. Set/reset/create enforce 12 characters via assertPasswordStrong.

Two refresh hashes on User allow desktop + phone. A third login replaces the older slot. Password change clears both and access JWTs die after passwordChangedAt (jwt.strategy compares payload.iat).

Users: Settings → Users is Super Admin only (user:manage). Creating an IT Admin or another Super Admin happens there. Linking a user to an employee row is how Super Admin/IT staff raise tickets as themselves; Super Admin can also lazy-provision an Employee on first ticket raise.

4.2 Tenancy & platform

[backend/src/tenancy/](#) · [frontend/src/pages/settings/workspace.tsx](#)

provision.ts creates a tenant + Super Admin + starter catalog (ticket categories, asset categories including MOU/HDS/KEY/CAM/DOCK, default approval matrix). sample-company.ts loads ~25 laptops for a trial — never the 1,250-row internal seed.

Tenant HTTP: GET /api/tenant snapshot, PATCH branding, onboarding skip/sample, CSV templates, export, billing, DELETE close-account (confirm name + password).

4.3 Assets (serialized hardware)

[backend/src/assets/](#) · [frontend/src/pages/assets/](#)

This is the system's core entity. List filters: status, location, category, department, q, unaudited. Actions are dedicated endpoints rather than a generic PATCH of status, so each path can write the right assignment/transfer row and audit action.

Endpoint idea	Effect
POST /assets/:id/assign	status assigned, AssetAssignment row, optional accessoryIds[], optional expectedReturnAt (loaner; never auto-returns)
POST /assets/:id/transfer	location and/or assignee change; AssetTransfer history; stays assigned if an employee remains
POST /assets/:id/retire	retired + reason
POST /assets/:id/status	other allowed transitions via lifecycle.ts
POST /assets/bulk	status transfer retire assign; per-id partial success
POST /assets/:id/audit and /assets/audit-by-code	physical audit stamp; public scan stays read-only
GET /assets/:id/qr	PNG sticker
DELETE /assets/:id	Super Admin, and only if retired/disposed

Finance fields on an asset are redacted for non-admins. Employee lists are scoped to own assigned kit ("My kit" also includes accessory checkouts). Managers see team-scoped assets.

4.4 Employees, locations, departments, categories

Employees carry employmentType (permanent/contract/intern/consultant), joining date (required, default today, never createdAt), probation (default 90 days from DOJ), lastWorkingDate on offboard, managerId self-relation. List/profile RBAC: IT sees all; managers see self + direct reports; employees see self.

Offboard (POST /employees/:id/offboard) returns or reassigns assets, checks in accessories, sets isActive false, deactivates the linked User, keeps history. Hard DELETE is blocked when history exists. GET

/employees/:id/history merges assignments, transfers, accessories, consumables, requests, maintenance, audit — not a truncated slice.

Locations are full CRUD for IT Admin+. Departments and categories are Settings tabs (needed so first-run Welcome links are not dead). Hardware house is three URLs (/assets, /accessories, /consumables) plus in-page HardwareTabs; sidebar keeps three items so Playwright locators still work.

4.5 Accessories, consumables, issue kits

Accessories are reusable quantity stock (checkout/check-in). Consumables decrement on issue and fire low_stock notifications. They are not fake serialized assets — do not invent AST codes for mice. Prompt 38 added brand/model/location/threshold and checkout serial/parent asset/due; duplicate serials warn, they are not unique-constrained (accessories are still quantity lines).

Issue kits are named checkout recipes: next available asset in a category (+ optional location) plus accessory IDs. Settings CRUD; used from employee runbooks / assign flows.

4.6 Asset requests

See flow §5.4. Frontend /requests. IT Support can fulfill (Prompt 32) so the nav item is not ornamental.

4.7 Maintenance

[backend/src/maintenance/](#) · [frontend/src/pages/maintenance.tsx](#)

Hardware repair on one asset — separate from Support Tickets (software/network/access) and from Requests (wanting new kit). Employees may report only on assets assigned to them. IT manages the queue. Transitions live in maintenance-status.ts and drive the asset's status in the same Prisma transaction.

Prompt 35 OEM claim fields live on AssetMaintenance: coverage, oemCaseId, rmaNumber, claimInvoiceNo, incidentKind. Filing copies serial + invoice into notes, moves reported → under_repair, and sets loanerNeeded when the assignee has no other device. Linked AMC/warranty contracts are read from VendorContractAsset.

vendorId is a real FK; IT Support picks names from GET /vendors/options (no bank fields).

4.8 Support tickets (helpdesk)

[backend/src/tickets/](#) · [frontend/src/pages/tickets/](#)

Spiceworks-simple: categories with default priorities, templates, canned responses (optional statusOnSend), public vs internal comments, watchers, time logs, attachments (bytea, 8 MB, shared allow-list), optional asset, duplicate-of (one-way close + system comment, not a merge), CSAT 1–5 once on resolve, bulk assign/close, CSV/PDF, first-response SLA chips, waiting_on_employee, Portal vs Email channel.

RBAC: any employee creates and sees own + watched (public comments). IT Support/Admin/Super Admin manage the queue. Managers see own + direct reports; no assign/internal/time. Auto-assign is least-loaded active IT Support unless autoAssign === false.

Account playbook (Prompt 35): ticket records of M365/VPN/biometric resets — no Entra API. Identity verify is a timestamp; reset buttons stay gated on it.

4.9 Notifications & mail

MailerService.send never throws. Order: RESEND_API_KEY (HTTPS) → SMTP_HOST → console log. HTML templates in ticket-email-templates.ts share one inline-styled shell (logo, #0958D9, one CTA). Staff with emailNotifyPref=daily_digest skip immediate staff emails; requesters/watchers stay immediate. In-app notifications are always immediate.

Crons (all no-ops if the process is asleep — keep-alive exists for that):

Cron	When	What
warranty-threshold-alerts	Daily 08:00	90/60/30 remaining days, de-duped per asset+threshold
warranty-weekly-digest	Monday 08:00	IT digest
ticket-daily-digest	Daily 08:00	Staff who opted into digest
ticket-overdue-mail	Hourly	Requester once (overdueMailedAt)
email-in-poll	Every minute	IMAP unseen mail, if IMAP_HOST set
contract-renewal-alerts	Daily 08:00	90/60/30/7
prune-auth-audit	Daily 03:00	Old login/auth_failure rows

4.10 Dashboard, search, reports

GET /dashboard/metrics, /setup, /by-location, /trends, /warranty-expiring, /attention. IT Admin keeps KPI tiles; IT Support’s home is the ordered My work list (overdue tickets, unassigned, waiting 3+ days, stale repairs, incomplete checklists, contracts ending 14d, warranties 14d, joining/probation). Low-stock and to-fulfill stay on the API for sider badges but are not mixed into that morning list.

GET /dashboard/setup is Super Admin / IT Admin only (Prompt 32). freshInstall is true only when assets, employees, and locations are all zero — seeded demo never shows Welcome.

Global search GET /api/search?q= feeds the 𐌆𐌀𐌆𐌀 command palette (header search field only opens the palette). EMP-... queries are exact. Reports: asset/employee/location/warranty/supplies plus procurement-* variants; CSV via ExcelJS, PDF via PDFKit landscape.

4.11 Import, saved views, reconciliation, webhooks, QR

See flows §5.7–5.8. Webhooks: Settings → Webhooks. Events asset.created and asset.status_changed. HMAC-SHA256 of the raw JSON body in X-NewVision-Signature; secret shown once. Delivery awaited in-request with 2.5s timeout; failures recorded, never thrown to the caller.

4.12 Notes & manual correction

GET/POST /api/notes?entityType=&entityId= append-only. Past occurredAt is tagged Backfilled. POST /api/records/:entityType/:id/manual is Super Admin/IT Admin, mandatory reason, per-entity field whitelist, audit action=manual_override. Procurement fields are narrow: no status, totals, or relational ids (those have dedicated workflows). Employee self-edit of phone/title and requester ticket subject/description are regular update audits, not overrides.

4.13 Procurement

[backend/src/procurement/](#) · [frontend/src/pages/procurement/](#)

See flow §5.5. Constants: PROCUREMENT_MATCH_TOLERANCE_PCT default 2. Unique (vendorId, invoiceNumber); Super Admin may set correction: true to store INV-...-CORR. GSTIN must match PAN when both present.

4.14 Team Chat

[backend/src/chat/](#) · [frontend/src/pages/chat/](#)

Staff-only. Seeded channels #it-ops, #helpdesk, #procurement. DMs keep Prisma type dm (UI label Direct). Mentions are [@Name](mention:id) plus @channel/@here. Sender can edit/soft-delete; Super Admin/IT Admin can delete any message; IT Support cannot moderate others. Read receipts only on DMs/groups with ≤8 members. Compact vs Comfy density in localStorage nv.chat.density.

4.15 Help

In-app documentation at /help, modelled on MkDocs Material structure but using NewVision tokens (not ING colours). Content is help/articles.ts plus employeeHowtos.ts (12 static “For employees” articles). Screenshots: cd frontend && npm run screenshots. Help markdown links must stay underlined (axe link-in-text-block).

4.16 Frontend shared components worth knowing

- [DataGrid](#) — default table. Column layout persists in localStorage per tableKey. Overflow cells get tooltips.
- [KpiCard](#) [StatusTag](#) [TicketStatusTag](#) [SlaChip](#) — never use AntD preset gold/green tags; they fail WCAG. Explicit pairs live in theme.ts / TicketStatusTag.tsx.
- [CommandPalette](#) — ⌘K / Ctrl+K / header search / shortcut / .
- [FirstRunWelcome](#) — empty estate only.
- [RecordNotes](#) [ManualEdit](#) — notes rail + override modal.
- [HardwareTabs](#) [EmployeeBottomNav](#) [DesktopOnlyBanner](#) — Prompt 38 chrome.
- [ScreenshotPasteZone](#) — tickets and chat; real files beat Office thumbnails.

5. Data & Process Flows

Each flow is one diagram plus the actual code path. If you change a status without going through the named endpoint, you will desynchronize history tables and audit rows.

5.1 Login and session restore

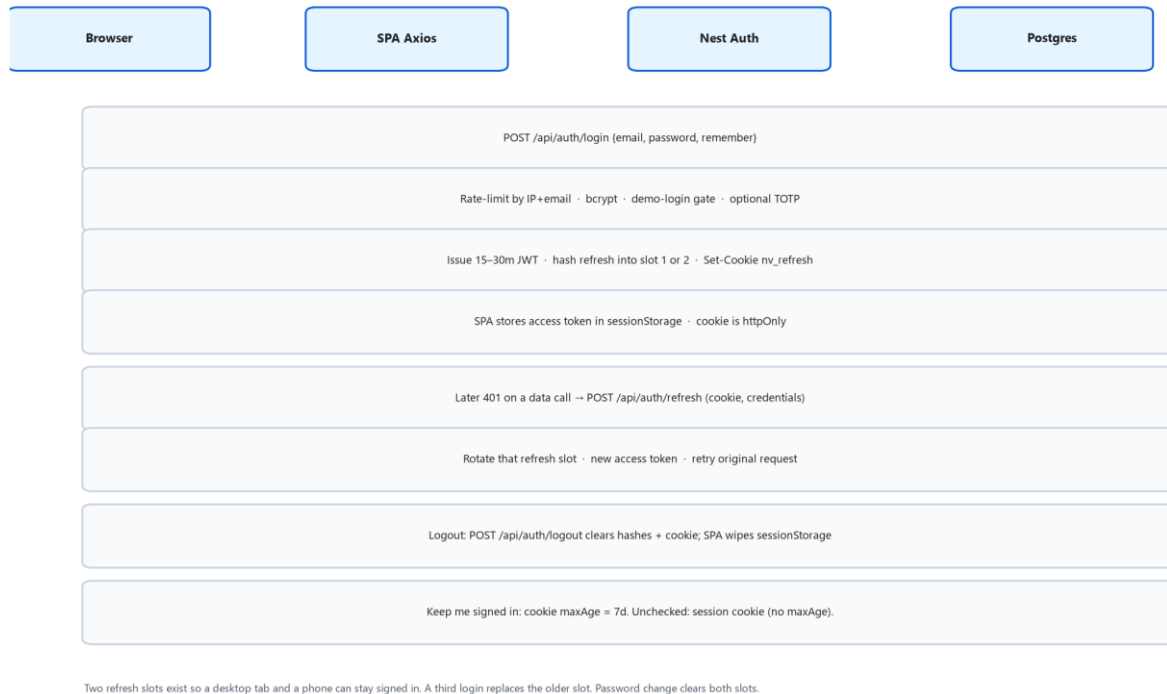


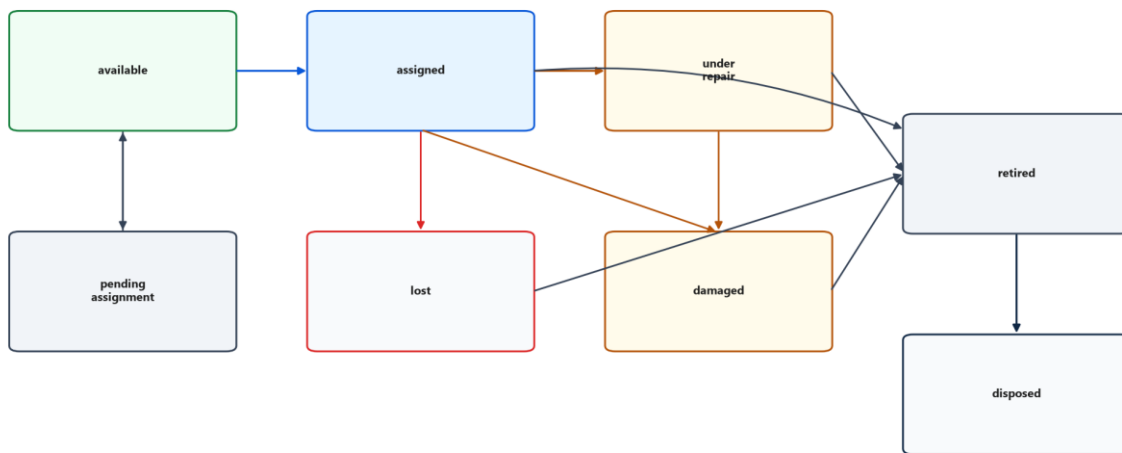
Figure 4. Login, refresh cookie, and Axios 401 retry.

1. User POSTs /api/auth/login. LoginRateLimitService (in-memory, 8 failures / 15 minutes / IP+email) may 429.
2. validateUser runs unscoped findUnique by email, bcrypt, isActive, demo-login gate (blocked in production unless ALLOW_DEMO_LOGINS=true).
3. Optional TOTP gate (mfaGate). REQUIRE_SUPERADMIN_MFA is currently commented out / false on Render so demo Super Admin can sign in without TOTP — Unclear when ops will re-enable it.
4. issueTokenPair signs access JWT (JWT_EXPIRES_IN, default 30m) and refresh JWT (7d, JWT_REFRESH_SECRET or JWT_SECRET::refresh). Hash stored on User slot 1 or 2.
5. Set-Cookie nv_refresh; httpOnly; Secure+SameSite=None in production (cross-site web.onrender.com → api.onrender.com); Lax locally. path=/api/auth. maxAge 7d if remember, else session cookie.
6. SPA writeSession to sessionStorage. On later 401, tryRefresh posts to /auth/refresh with credentials, writes the new access token, retries once.

7. `restoreSession()` on boot: if this tab has no access token, try the cookie (Keep me signed in).

```
export const REFRESH_COOKIE = 'nv_refresh';
// path: '/api/auth' httpOnly: true
// production: secure true, sameSite 'none' // SPA and API are different hosts
// local:      secure false, sameSite 'lax'
```

5.2 Asset lifecycle (assign / transfer / retire / repair)



available also → under_repair (spare in storage), lost → available or retired, damaged → under_repair, available, or retired. Transfer is an action, not a status.

Same-status is idempotent, disposed is terminal. Hard-delete is Super Admin only on retired/disposed.

Figure 5. Allowed AssetStatus transitions (lifecycle.ts). Transfer is an action, not a state.

Happy path — issue a spare laptop:

1. IT Admin opens a row with status available (or uses an Issue kit).
2. `POST /api/assets/:id/assign { employeeId, accessoryIds?, expectedReturnAt? }` inside a transaction: `assertTransition(available → assigned)`, create `AssetAssignment`, optional `AccessoryCheckout` rows, audit action=assign, webhook `asset.status_changed`, `tenant.recordAssignment` for onboarding.
3. Later floor move: `POST /transfer { toLocationId and/or toEmployeeId, reason }`. History row in `asset_transfers`. Status stays assigned if someone still holds it.
4. End of life: `POST /retire { reason }` → retired. Super Admin may DELETE only from retired/disposed.

Illegal transitions throw `InvalidTransitionError` → 400 with the allowed list. Same-status is allowed (idempotent). Dashboard KPIs and reports count these statuses; do not add a new status without updating `lifecycle.ts`, the Prisma enum, `StatusTag`, and seed.

5.3 Maintenance / OEM repair

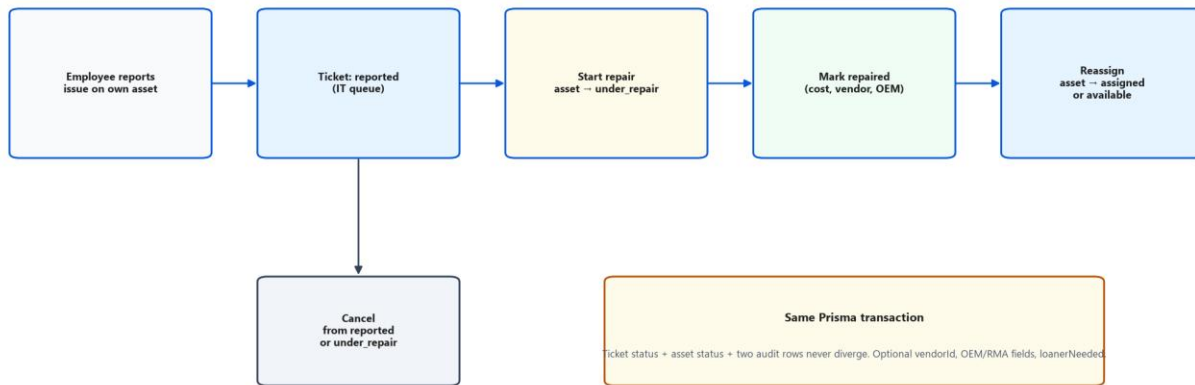


Figure 6. Repair ticket and coupled asset status in one transaction.

1. Employee (or IT) creates a maintenance ticket: reported. Employee path requires the asset assigned to them.
2. IT Start Repair: ticket `under_repair` AND asset `under_repair`, same transaction, two audit rows.
3. Mark Repaired: actual cost, vendor, dates, OEM fields. Ticket repaired.
4. Reassign: ticket reassigned; asset returns to assigned (if still has an employee) or available.
5. Cancel is reachable from reported or `under_repair` and must restore the asset accordingly.

5.4 Asset / accessory request

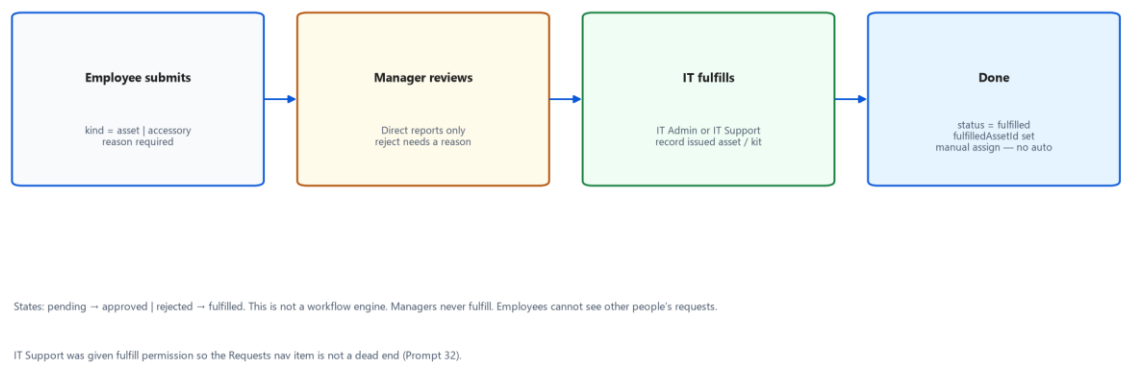
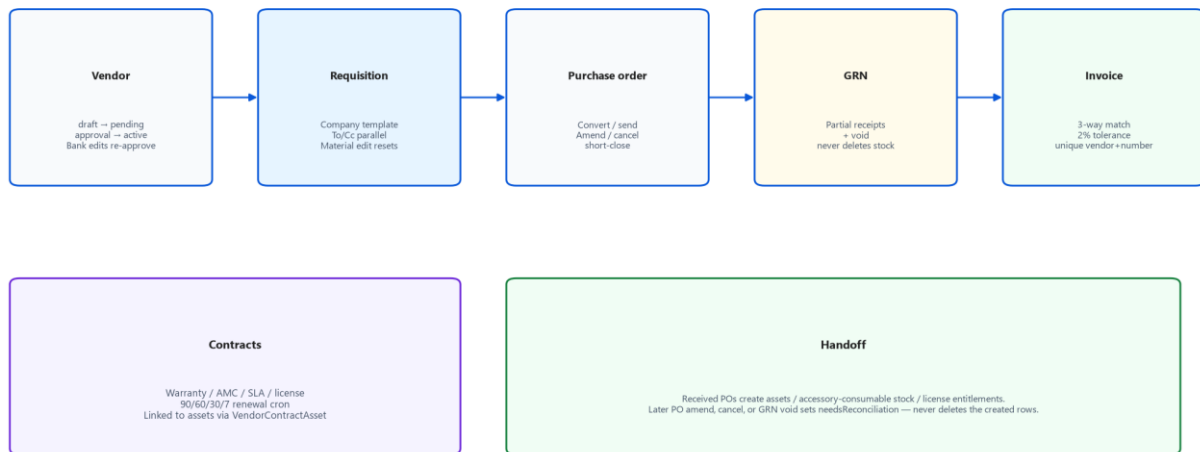


Figure 8. Single-step request approval. Not a workflow engine.

pending → approved|rejected → fulfilled. Rejection requires rejectionReason. Fulfillment records fulfilledAssetId or a kit; it does not auto-pick an asset. Manager scope is direct reports only.

5.5 Procurement



RBAC: Super Admin / IT Admin own Vendors, POs, Contracts. Managers raise and see team requisitions only. IT Support gets /vendors/options (names, no bank) for repair pickers.

Team plan module. Trial tenants must finish or skip the first-hour checklist before procurement APIs open.

Figure 9. Vendor → requisition → PO → GRN → 3-way match invoice, plus contracts and handoff.

Why the complexity exists: this matches the company's real To/Cc approval email, not an abstract BPMN engine.

1. Vendor lifecycle draft → pending approval → active (or suspended/blacklisted). Bank-detail edits on an active vendor go to bankChangePending rather than applying instantly.
2. Requisition copies a matrix rule (parallel by default, sequential if the rule says so). Required approvers = every active user with the matching role at that threshold. Material edits (vendor, category, type, tax, total, any line) reset the chain and increment revision. Title/dates/locations do not.
3. Convert to PO, mark sent, amend (history rows), cancel or short-close.
4. GRN may be partial; void is allowed. Void/amend/cancel flags needsReconciliation on auto-created assets/handoffs instead of deleting them.
5. Vendor invoice: 3-way match vs PO and GRN within 2%. Exception invoices need a note before payment can be approved. Unique (vendorId, invoiceNumber).
6. Contracts 90/60/30/7 renewal alerts; renew/clone. Scorecards live alongside.

5.6 Support tickets and email-in

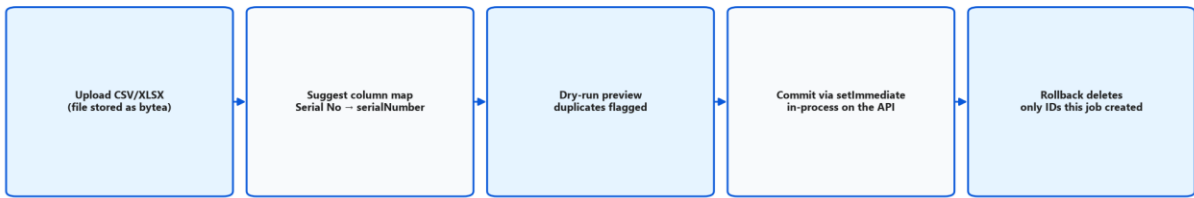


Figure 7. Portal vs email-in, matching rules, and lifecycle side effects.

Inbound matching is strict on purpose (DECISIONS.md Prompt 20): In-Reply-To/References first, then [TCK-000123] in the subject. Unrecognized From still creates a ticket flagged unmatchedSender — the system never auto-creates an Employee from a stranger’s email. HTML is sanitized; MIME attachments go through assertAllowedUpload into TicketAttachment.

Outbound ticket mail sets Message-ID / In-Reply-To / References, Reply-To HELPDESK_MAILBOX, and the ticket number in the subject so replies round-trip.

5.7 Import jobs and reconciliation



Synchronous POST `/api/import/assets|employees` still exists for small files. Jobs live under Settings, not a new sidebar item.

Reconciliation is a separate manual CSV set-diff (employeeCode/email or assetCode/serialNumber) — not live AD/HR sync.

Figure 12. Background import without Redis.

Tests call `process()` directly to avoid racing `setImmediate`. Rollback deletes only IDs recorded on that job (and clears dependent assignment/transfer/maintenance/notification rows first so FKs do not block). Synchronous `/import/assets` and `/import/employees` remain for small files.

Reconciliation is a case-insensitive set-diff, cap 500 finding rows per side stored, exact counts. Match keys: `employeeCode/email` or `assetCode/serialNumber`. Manual upload only.

5.8 QR scan and scan-to-audit

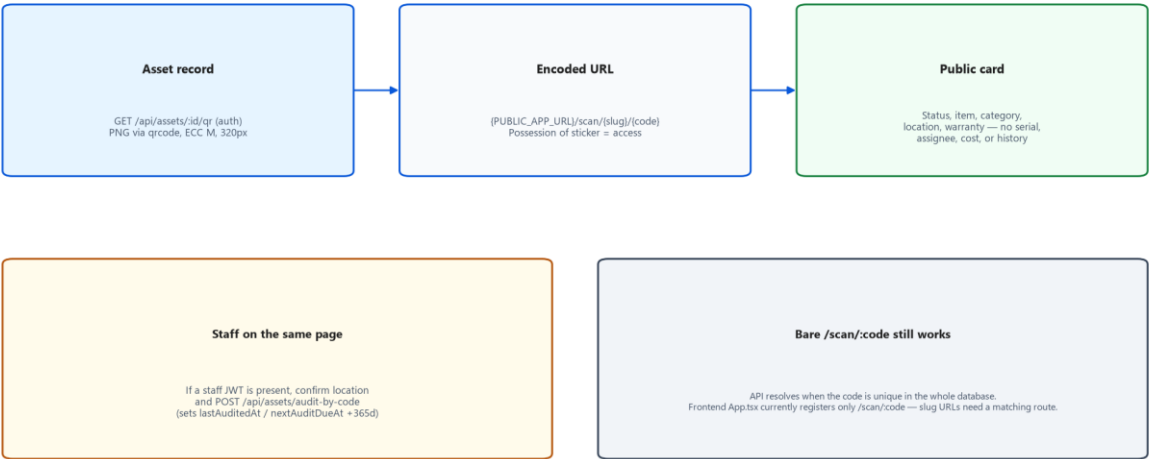
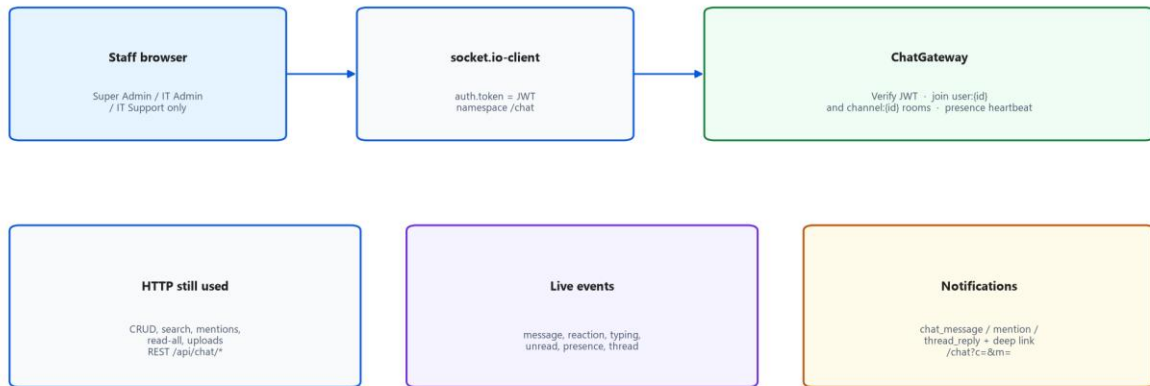


Figure 13. Sticker URL, public card, and staff audit.

Physical possession of the sticker is the access control. The public JSON omits serial and assignee (Prompt 20) even though older docs still mention them — trust the controller present() method. Staff with a JWT on the same page can confirm location and POST /assets/audit-by-code. Dashboard unaudited attention includes never-audited; My work does not (the seed pile would drown the list).

Routing caveat. scanPageUrl() prefers /scan/{slug}/{code}. ScanPage reads useParams().slug. App.tsx currently declares only /scan/:code. Tenant-scoped sticker URLs will not bind slug until a second route is added. Bare /scan/:code still works when the asset code is globally unique.

5.9 Team Chat realtime



Employees and Managers cannot open /chat (API 403 + RoleRouteGuard). Chat is a Team-plan module. Presence is in-memory (10s heartbeat, prune 25s) — not a lock server.

/chat is not inside ThemedLayout (same pattern as Help) so there are not two left rails.

Figure 11. HTTP CRUD plus Socket.IO /chat namespace.

On connect the gateway verifies JWT, sets `client.data.userId/tenantId`, joins `user:{id}` and every `channel:{id}` membership, updates `lastSeenAt` via `updateMany` (not `update` — a missing user after a seed wipe must not throw P2025). Presence is in-memory. HTTP remains the source for history, search, mentions, read-all, and uploads.

5.10 Signup (API) and first hour

POST `/api/auth/signup` provisions a tenant, Super Admin, starter catalog, 14-day trial. The SPA pages `signup.tsx` and `trust.tsx` exist and `login.tsx` links to them, but `App.tsx` does not register `/signup` or `/trust`, so an unauthenticated click currently hits the Authenticated layout and redirects to `/login`. The API itself is ready; adversarial isolation is covered by `backend/test/prompt37-tenancy.e2e-spec.ts`.

First-hour checklist: import employees, import assets, assign one, scan a QR, resolve a ticket — or skip. POST `/api/tenant/onboarding/sample` loads the small sample company.

5.11 Webhook delivery

On asset create or actual status change, `WebhooksService` POSTs JSON to each subscriber with headers `X-NewVision-Event` and `X-NewVision-Signature` (HMAC-SHA256). 2.5s `AbortSignal.timeout`. `lastError` / `lastStatus` stored on the endpoint row. Failures never fail the user-facing mutation.

5.12 Offboarding

POST /employees/:id/offboard requires lastWorkingDate (Prompt 38). Returns assigned assets (or reassigns if asked), checks in open accessory checkouts, deactivates employee + user, writes audit. Consumable issue rows stay (historical). Linked login can no longer authenticate (isActive false).

6. Configuration & Environment

Backend reads process.env via ConfigModule.forRoot({ isGlobal: true }) and dotenv at main.ts. There is no extra config YAML. Frontend has a single Vite env: VITE_API_URL (and production VITE_SHOW_DEMO on Render).

6.1 Backend variables

Source of truth for names: backend/.env.example, docker-compose.yml, render.yaml, and the grep of process.env in src/. If a variable is missing, the last column says what actually happens.

Variable	Controls	If missing / wrong
DATABASE_URL	Postgres connection (Prisma CLI + pg Pool)	Process cannot start / migrate. prisma.config.ts throws if unset when the CLI loads.
JWT_SECRET	Access token HMAC	Production bootstrap throws. Dev falls back to 'dev-only-secret-change-me' — never ship that.
JWT_REFRESH_SECRET	Refresh token HMAC	Falls back to JWT_SECRET + '::refresh'. Use a distinct value in production.
JWT_EXPIRES_IN	Access TTL	Defaults to 30m in AuthModule. .env.example says 15m; compose/Render set 30m. Assumed: 30m is what local Docker and prod actually run.
PORT	Listen port	3000
CORS_ORIGIN	Allowed SPA origins (comma-separated), Socket.IO origin	http://localhost:5173. Wrong value = browser CORS errors and chat sockets fail. Production must be the Render web URL.
NODE_ENV	production gates, swagger default, cookie flags, demo logins	Not production → swagger on, demo logins on, refresh echoed in login JSON, cookies not Secure.
ALLOW_DEMO_LOGINS	Whether @newvision.local accounts may authenticate	Unset: allowed when NODE_ENV !== production. Production blocks them unless this is 'true'. Render currently sets true for the demo.

SWAGGER_ENABLED	Mount /api/docs	Unset: on unless production. Render sets false.
SEED_ON_START	Entrypoint runs npm run seed	false → migrate only. true without SEED_IF_EMPTY wipes demo data every boot (dashboard seedWipeRisk banner).
SEED_IF_EMPTY	Seed no-ops once any user exists	Protects production. Compose and Render set true.
SEED_MODE	bootstrap vs demo estate	bootstrap + empty DB → one Super Admin, no 1,250 assets. Currently commented out on Render; seed.ts bootstrap function kept. Unclear whether live DB still has demo users (PROGRESS says yes).
BOOTSTRAP_ADMIN_EMAIL / PASSWORD / NAME	First Super Admin for bootstrap	Entrypoint exits non-zero if bootstrap seed fails. Password must be ≥12 chars.
RESEND_API_KEY	HTTPS transactional mail	Mailer tries SMTP next, then console. Live inbox unproven without this.
SMTP_HOST/PORT/USER/PASS	SMTP fallback	Ignored if Resend key set. Render Free cannot open 587/465.
MAIL_FROM	From header	Default 'NewVision IT <it-noreply@newvision.local>' — not a deliverable sender.
HELPDESK_MAILBOX	Reply-To and loop detection	Falls back to MAIL_REPLY_TO then it@newvision.local
IMAP_HOST/PORT/SECURE/USER/PASS/MAILBOX	Inbound poller	If IMAP_HOST blank, cron no-ops; app still starts. Webhook ingest can still work.
EMAIL_INGEST_SECRET	X-Email-Ingest-Secret for POST /email-in/webhook	Compared timing-safe. Unset → webhook rejects. Super Admin/IT Admin can still POST /email-in/ingest.
PUBLIC_APP_URL	QR URLs + email CTA/logo	http://localhost:5173. Wrong host = stickers and email buttons point at localhost in production.
HOSTING_REGION	GET /api/health region + residency note	Defaults singapore
PLATFORM_ADMIN_SECRET	Ops billing hook	Wrong/missing → { ok: false }
REQUIRE_SUPERADMIN_MFA	Force TOTP for Super Admin	Code currently commented; Render sets false. Assumed: TOTP enroll endpoints still exist.

TOTP_ENCRYPTION_KEY	Encrypt TOTP secrets at rest	Falls back to JWT_SECRET
PROCUREMENT_MATCH_TOLERANCE_PCT	3-way match	Default 2
FORCE_LOGIN_RATE_LIMIT	Tests	NODE_ENV=test skips throttle unless this is true

6.2 Frontend variables

Variable	Controls	If missing / wrong
VITE_API_URL	Axios base URL (baked in at Vite build time)	Defaults to http://localhost:5173's API at http://localhost:3000/api. If you pass an origin without /api, axios.ts appends /api. Production: fromService RENDER_EXTERNAL_URL of the API. If the UI was built before the API URL existed, trigger a Manual Deploy of newvision-web.
VITE_SHOW_DEMO	Render sets 'true'	Intended to show demo affordances; login demo box itself is gated on import.meta.env.DEV, so production UI does not print Password123! unless you are in Vite dev. Unclear how much VITE_SHOW_DEMO still changes besides being present in render.yaml.

6.3 Compose vs Render vs local .env

Three files intentionally disagree because they target three environments:

- **backend/.env.example** — developer laptop talking to local Postgres. JWT_EXPIRES_IN=15m, ALLOW_DEMO_LOGINS=true, SWAGGER on, IMAP blank.
- **docker-compose.yml** — JWT 30m, SEED_ON_START=true, SEED_IF_EMPTY=true, CORS http://localhost:5173, no mail keys.
- **render.yaml** — production NODE_ENV, generated JWT secrets, ALLOW_DEMO_LOGINS currently true (demo), SWAGGER false, REQUIRE_SUPERADMIN_MFA false, Resend/IMAP sync:false (you paste in dashboard), duplicate JWT_REFRESH_SECRET key appears twice in the YAML (last wins; harmless but sloppy).

6.4 Secrets that must never be committed

- JWT_SECRET, JWT_REFRESH_SECRET, PLATFORM_ADMIN_SECRET, BOOTSTRAP_ADMIN_PASSWORD, SMTP_PASS, IMAP_PASS, RESEND_API_KEY, EMAIL_INGEST_SECRET, TOTP_ENCRYPTION_KEY.
- backend/.env and frontend/.env are local. Rotate live Render secrets if the demo password Password123! is still valid on the public URL.

- Webhook signing secrets are shown once at create time; they are not recoverable from later GET.

7. Setup & Local Development

Two supported ways to run. Docker is the path that matches CI most closely. Node 22+ is required (developed on Node 24; CI uses 22). PostgreSQL 16.

7.1 Docker Compose (recommended)

Requires Docker Desktop. From the repo root:

```
docker compose up --build
```

Service	URL	Notes
frontend	http://localhost:5173	Vite inside the frontend container, --host 0.0.0.0
backend	http://localhost:3000/api	Swagger at /api/docs
postgres	localhost:5432	user/password/db all newvision

On first boot the API endpoint runs prisma migrate deploy then npm run seed because SEED_ON_START=true. With SEED_IF_EMPTY=true a second start will not wipe users. If you need a clean demo estate: docker compose down -v (destroys the pgdata volume) then up again.

Reseed without destroying the volume:

```
docker compose exec backend npm run seed
```

All demo passwords are Password123! (12 characters, meets the new minimum):

Role	Email
Super Admin	superadmin@newvision.local
IT Admin	itadmin@newvision.local
IT Support	support@newvision.local
Manager	manager@newvision.local
Employee	employee@newvision.local

The seed builds a realistic Pune / Hyderabad / Bhopal estate (~1,180 employees, ~1,250 assets) plus accessories, consumables, tickets, chat channels, and procurement samples. Chat Playwright pings are cleaned up so the demo timeline is not full of “Prompt24 ping”.

7.2 Local Node (no Docker for the app)

Create two databases: newvision and newvision_test. Then:

```
cd backend
cp .env.example .env          # set DATABASE_URL if your Postgres is not newvision/newvision
npm install                   # .npmrc already has legacy-peer-deps=true
npx prisma generate
npx prisma migrate deploy
npm run seed
npm run start:dev             # tsc -w + node --watch dist/main.js → :3000
```

```
cd frontend
npm install
npm run dev                   # :5173
# frontend/.env already has VITE_API_URL=http://localhost:3000/api
```

start:dev compiles with tsc -w rather than nest start --watch. That is intentional (DECISIONS.md): the project does not depend on @nestjs/cli at build time.

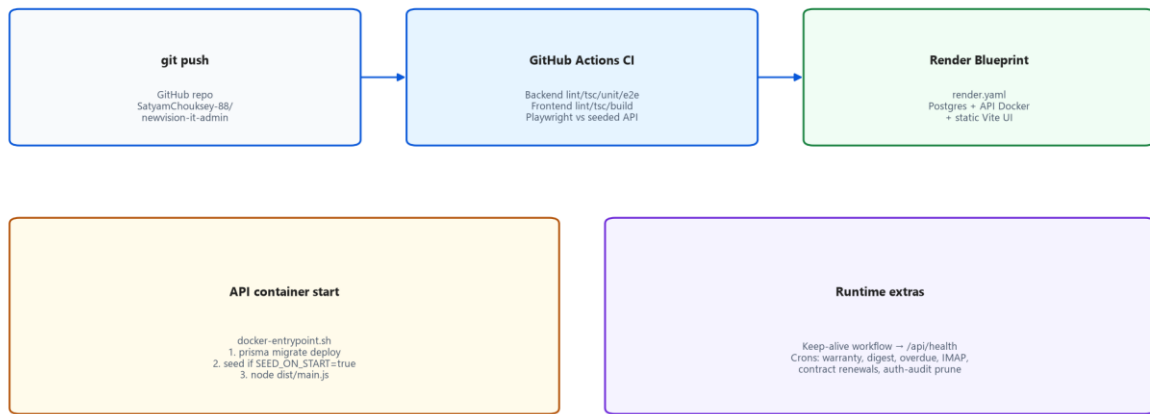
7.3 First-run empty estate

If you migrate without seeding, there are no logins. Either run the demo seed, or SEED_MODE=bootstrap with BOOTSTRAP_ADMIN_* to create one Super Admin. The Welcome card appears only when locations, employees, and assets are all zero. Settings → Categories and Employees → Add employee exist so those Welcome links work.

7.4 Common local pitfalls

- Windows CRLF on docker-entrypoint.sh — the image sed-strips CR; .gitattributes also forces LF on *.sh.
- Prisma generate needs DATABASE_URL even though it does not connect. The Docker build stage sets a throwaway URL.
- E2E tests need the seeded API on :3000. Playwright starts Vite itself (reuseExistingServer locally).
- If login works but every list is empty, you are probably on a trial tenant with no sample data — load sample or seed.
- CORS: frontend origin must match CORS_ORIGIN exactly, including http vs https and no trailing slash surprises.
- Cookie SameSite=None requires Secure; localhost uses Lax so refresh works without HTTPS.

8. Build, Test & Deployment



Frontend Dockerfile in compose is the Vite dev server (hot reload), not the production static build. Production UI is Render static: npm ci && vite build, SPA rewrite to index.html.

Hobby Postgres expires after 30 days on the free plan. Do not recreate the instance to change region — that drops data. Singapore is demo; India on request.

Figure 14. CI, container start, and Render.

8.1 Build

Backend: `tsc -p tsconfig.build.json → dist/main.js`. Not nest build. Runtime image copies `dist + src + prisma + node_modules` (including `devDeps`) because `seed.ts` imports RBAC from `src` and needs `tsx/prisma CLI` in the container.

Frontend: `vite build`. `chunkSizeWarningLimit 1300` because Ant Design's used subset is ~1.2 MB and cannot be split without circular chunks. Charts are a separate chunk so login does not download G2.

8.2 Lint and typecheck

```
cd backend && npm run lint && npm run typecheck
cd frontend && npm run lint && npm run typecheck
```

Biome, not ESLint. `npm run lint:fix / format` exist. Do not add `typescript-eslint` — it throws on TypeScript 7.

8.3 Backend unit tests

`npm test` in `backend/`. Prisma is mocked. Specs live next to the code (`lifecycle.spec.ts`, `permissions.spec.ts`, `column-map.spec.ts`, ...). They import `describe/it/expect` from `@jest/globals`. Transform is `@swc/jest` with legacy decorators.

Last recorded count in PROGRESS (Prompt 27/32 era): 101 unit tests. Re-run rather than trusting the number — it drifts every prompt.

8.4 Backend API e2e

npm run test:e2e — Jest config test/jest-e2e.js, runInBand, forceExit true because ScheduleModule cron handles stay open. globalSetup creates newvision_test and migrate deploy. Each suite seeds a small fixture. Needs Postgres reachable (CI service container or local).

Suites include auth, assets, phase2–4, tickets, ticket-emails, email-in, chat, procurement, tenancy (prompt37), security (prompt32), join-kit (prompt38), OEM claim, GSTIN, invoices, and more under backend/test/*.e2e-spec.ts.

Known flake. A huge overnight run starved phase2.e2e-spec.ts beforeAll after many hours. Re-running that file alone passed. Prefer runInBand and do not treat a hung 16h job as a product regression.

8.5 Playwright

Backend must already be seeded on :3000. Then:

```
cd frontend
npx playwright install chromium # first time
npm run test:e2e
npm run test:e2e:report
```

Config: workers 1, fullyParallel false, 45s test timeout, Chromium only, webServer npm run dev. README's "103 test() cases" is a grep count, not a last-green claim. QA_RESULTS.md is the last executed subset (Help + axe on 2026-09-14).

Accessibility: @axe-core/playwright. Serious/critical fails the test. Ant Design dropdown/modal portals and .ant-table-measure-row are excluded (upstream). aria-hidden-focus is re-enabled.

8.6 CI

.github/workflows/ci.yml on every push and pull_request, Node 22:

1. Backend job: npm ci, prisma generate, lint, typecheck, unit, test:e2e against Postgres 16 service (user postgres / db newvision; globalSetup creates newvision_test).
2. Frontend job: npm ci, lint, typecheck, build.
3. Playwright job (needs both): migrate+seed+build backend, nohup node dist/main.js, wait on /api/docs (note: that wait will fail if SWAGGER is off — CI does not set NODE_ENV=production, so swagger is on), install Chromium, npm run test:e2e, upload playwright-report artifact.

8.7 Production deploy (Render)

1. Push to GitHub. Dashboard → New → Blueprint → this repo, or let the existing services auto-deploy on path filters (backend/** vs frontend/**).
2. API Dockerfile build: prisma generate (dummy DATABASE_URL) + tsc. Runtime: openssl, entrypoint migrate deploy, seed if SEED_ON_START, node dist/main.js. healthCheckPath /api/health.
3. Web: rootDir frontend, npm ci && npm run build, staticPublishPath dist, SPA rewrite /* → /index.html, security headers including CSP connect-src self + https://*.onrender.com.
4. Set secrets in the dashboard (Resend, IMAP, mailbox, ingest secret). Do not copy local .env.
5. If the UI calls the wrong API host, Manual Deploy newvision-web so Vite rebuilds with the live VITE_API_URL.
6. Keep-alive: .github/workflows/keep-alive.yml curls https://newvision-api.onrender.com/api/health every 10 minutes. Safe to delete on a paid always-on instance. render.yaml currently sets the API plan to starter (always-on); comments in PROJECT_STATUS still talk about Free sleep — Assumed: starter is what ships now, keep-alive is belt-and-suspenders.

Backup drill: docs/BACKUP.md. App has no backup UI. pg_dump the Render database; restore onto a scratch instance; prove tenant isolation still holds.

9. Conventions & Patterns

Match these so a sixth prompt does not invent a seventh style. Almost every “why is it like this?” has a paragraph in DECISIONS.md — read that before undoing a constraint.

9.1 API design

- Global prefix api. Controllers are resource plural: assets, employees, support-tickets.
- List query: _start _end _sort _order plus domain filters and q. Response { data, total }. Headers x-total-count and sometimes X-Row-Count.
- Mutations that are really verbs (assign, transfer, retire, review, fulfill) are POST sub-routes, not PATCH of a status field.
- RBAC: @Roles(RoleName....) on each handler. ROLE_PERMISSIONS in permissions.ts is the informational matrix mirrored in frontend/src/access.ts and shown under Settings → Account. Keep them aligned with what @Roles actually grants.
- Public routes: @Public() on login, signup, refresh, health, forgot-password, public scan/QR, platform billing hook, email-in webhook.
- ValidationPipe whitelist + transform. DTOs with class-validator.
- Errors: Nest HTTP exceptions with human messages the SPA apiErrorMessage() can show.
- Audit every mutation via AuditService. Append-only; there is no edit/delete of audit rows (and that is a non-goal).

9.2 Prisma / tenancy

- Never query a tenanted model outside `runWithTenant` or a request that already went through `TenantInterceptor`, unless you wrap `runUnscoped()` and you know why (login, health, cron fan-out, public scan).
- Cron jobs must `forEachTenant` (or equivalent) so Prisma never runs a tenanted query with empty ALS.
- Raw SQL (dashboard trends) must add `tenant_id` by hand — the extension only wraps Prisma queries.
- IDs are autoincrement ints. `createdAt` / `updatedAt` on models. `snake_case @map` names.
- Do not add a new unique on email-like fields without thinking about global vs per-tenant. `User.email` is global; `Employee.employeeCode` is per tenant.

9.3 Frontend

- New screens: a `pages/<resource>/` folder, a Refine resource in `App.tsx`, a nav item in `navForRole`, and `@Roles` on the API. Missing any one of those is how employees see a button that 403s.
- Lists: `DataGrid`, not a one-off `Table`, unless you are the Accessories card grid (`Cards` is the default there; `Table` is the toggle).
- Colours: theme.ts tokens. Links #0958D9. Buttons/KPI #1677FF. Status chips: explicit pairs, never AntD preset gold/green.
- Light only. Do not add a dark theme. Force color-scheme: light.
- Copy IDs with `CopyButton`. Money is ₹. Warranty is “N days”, not a date dump without context.
- Route-level lazy + inner `Suspense`. Do not re-monolith the bundle.
- Help articles describe what actually works, including gaps. Recapture screenshots after chrome changes.
- Playwright uses `data-testid` where the mockup would otherwise hide a control (logout-button, logout-confirm).

9.4 Naming

- Prisma models `PascalCase`, tables `snake_case`. Enums already in schema (`AssetStatus`, `TicketStatus`, ...) — reuse them.
- Permission keys `subject:action` (`asset:assign`, `ticket:manage`).
- Frontend session keys `newvision:token`, `localStorage nv.*` for chrome prefs (`nv.chat.density`, `nv.dash.collapse.my-work`).
- Ticket numbers `TCK-000123`, employees `EMP-#####`, default assets `AST-PUN-LAP-0001`.

9.5 Uploads

One allow-list: `backend/src/common/uploads.ts assertAllowedUpload`. Blocks `html/svg/jar`; no `octet-stream` free pass. Shared by tickets, email-in, and chat. Do not invent a second list.

9.6 Tests

- Pure logic → unit spec next to the file. HTTP behaviour → `backend/test/*.e2e-spec.ts`. User-visible → `frontend/e2e/*.spec.ts`.
- Do not hit the demo database from Jest e2e; use `newvision_test`.

- Mailer assertions spy MailerService.send in e2e rather than a second harness.
- When you add a role-gated route, add both a 200 for the allowed role and a 403 for Employee.

9.7 Design system

Source of truth: design-reference/NewVision-standalone-src.html and DESIGN_TOKENS.md. An interim “futuristic” (glow/glass/bento) pass was explicitly reverted. Do not bring it back. Command palette and “Updated N minutes ago” were kept as functional, not stylistic, upgrades.

10. Known Issues / Technical Debt / Open Items

10.1 Must-know product/ops gaps

Item	Severity	Detail
Production mailbox unproven	Major / ops	QA D3. No RESEND_API_KEY / IMAP_* on Render at last QA. Code paths are tested; a real inbox is not. Do not mark E2 pass.
Production 5-role walk	Major / sign-off	QA D6. Local seeded app was tested; live Render 5-role + two-session chat was not this sitting.
Rotate live secrets	Major / security	PRODUCT_GAPS ledger: if production still has seeded Password123! and old JWT secrets, rotate. ALLOW_DEMO_LOGINS is currently true on Render.
Super Admin MFA off	Security trade-off	REQUIRE_SUPERADMIN_MFA=false and the enforcement code is commented so the demo admin can sign in. Re-enable before a real customer workspace.
Access token in sessionStorage	Documented residual XSS	Refresh is httpOnly. CSP+Helmet mitigate. A full cookie-only SPA rewrite was declined (Prompt 32).
Signup/Trust routes missing	Functional gap	Pages and API exist; App.tsx never mounts /signup or /trust. Unauthenticated links bounce to /login. Fix: add public routes next to /login and /scan/:code.
Tenant scan URL vs router	Functional gap	Backend prefers /scan/:slug/:code; App.tsx only has /scan/:code. ScanPage already reads slug.
Help article still says no public signup	Docs drift	frontend/src/help/articles.ts Getting Started still says there is no public self-service signup, which was true pre-Prompt 37.

Free Postgres 30-day expiry	Ops	render.yaml still lists plan: free for the database. Confirm in the Render dashboard; expired disks are unrecoverable. Do not recreate the instance to “move” region.
In-memory rate-limit & presence	Scale	Login throttle and chat presence die on process restart and do not share across replicas. Fine for one API container.
In-process import jobs	Scale	Large files block the event loop. Redis/Bull is a FUTURE_IDEAS item.
IMAP tenant inference	Edge	Poller infers tenant from recipient/from when ALS is empty. Multi-mailbox is a non-goal; one shared HELPDESK_MAILBOX is the design.

10.2 Test / process debt

- README, PROJECT_DOCUMENTATION.md, and PROGRESS.md contain stale test counts from many prompts. Count tests with grep or a fresh run.
- Full Playwright job is heavy; last QA did not re-run all specs. CI is supposed to, but path-filtered deploys and swagger wait are worth watching.
- Playwright create-asset tests accumulate demo rows; reseed to clean. Harmless.
- Jest e2e forceExit: true is a workaround for open cron handles, not a license to leak sockets in new code.

10.3 Stale documentation inside the repo

Treat as historical unless a section header says it was updated for Prompts 32–38:

- PROJECT_DOCUMENTATION.md — stops at Prompt 19 for architecture prose; later prompts are a footnote.
- Early DECISIONS.md bullets that still say Nest 12, AntD 6, JWT-only, no signup — superseded further down the same file.
- Help Getting Started “no public signup”.
- Comments in render.yaml about paused bootstrap vs README still teaching BOOTSTRAP_ADMIN_*. Live behaviour: SEED_IF_EMPTY skips if users exist; bootstrap keys are commented. Assumed the existing Render database still has demo accounts.

10.4 Deliberate non-goals (do not file as bugs)

See FUTURE_IDEAS.md. Highlights: SSO/SAML, live AD, CMDB, SLA engine, ticket merge/split, custom fields builder, PunchOut/OCR/GL/payments, Redis, dark mode, phone admin rewrite, AI triage, multi-mailbox/Slack ingest, chat calls.

10.5 Workarounds you will meet in the code

- `PrismaService` constructor returns a Proxy so `$extends` is what Nest injects. Do not “clean that up” without keeping tenant isolation.
- `runWithTenant` always async-enters ALS because a sync `als.run()` exited before Prisma’s microtask (login used to throw Missing tenant context).
- `findUnique without tenant` is allowed in the extension for the login race. Other operations throw.
- Chat presence updateMany instead of update to avoid P2025 after seed wipe.
- Ticket numbers TMP- then TCK- after insert because the id is the sequence.
- Public scan omits serial/assignee even though older comments/docs mention them.
- Ant Design measure-row checkboxes display:none so axe does not fail on hidden header clones.

11. Glossary

Term	Meaning in this codebase
NewVision / NewVisionITIS	Product name. Repo folder is IT_ADMIN. UI document title uses NewVisionITIS.
Estate	The set of serialized assets (+ accessories/consumables) for one tenant.
Tenant / workspace	One customer company. Row in tenants. Isolation key tenantId.
Starter vs Team	Plans. Team unlocks procurement, chat, maintenance. Trial is Team for 14 days, then Starter if unpaid.
IT seats	Count of users in SUPER_ADMIN / IT_ADMIN / IT_SUPPORT. Default cap 10. Assets are not metered.
My kit	Employee view of assigned assets + accessory checkouts. Serials on checkouts are IT-only.
My work	IT Support/Admin morning list (tickets/repairs/checklists/contracts/warranties), not the old mixed “needs attention” pile.
My IT	Employee shell (home + bottom nav). ROLE_CHIP label for EMPLOYEE.
Asset vs accessory vs consumable	Serialized device vs reusable quantity stock vs depletable quantity.
Issue kit	Named recipe: next available asset in a category + accessory IDs.
Maintenance vs ticket vs request	Repair this device vs helpdesk issue vs “please give me a device”.
Loaner	Assigned asset with expectedReturnAt. Never auto-checked-in; overdue is a My work nudge.

Scan-to-audit	Staff POST /assets/audit-by-code from the sticker page. Public card cannot do this.
3-way match	PO vs GRN vs vendor invoice, 2% tolerance.
Handoff	PO receipt creating assets/stock/licenses. Later upstream changes flag needs Reconciliation.
To/Cc approval	Requisition parallel approvers matching the company's email template.
Material edit	PR change that resets the approval chain (vendor/category/type/tax/total/lines).
Manual override	Super Admin/IT Admin field patch with mandatory reason. Not self-service typo fixes.
Canned macro	CannedResponse that may also set ticket status on a public staff reply.
unmatchedSender	Email-in created a ticket from an address that is not an employee.
duplicate-of	One-way close + comment. Not a merge.
CSAT	1–5 rating collected once on resolve; dropped if reopened.
nv_refresh	httpOnly refresh cookie, path /api/auth.
ALS	AsyncLocalStorage tenant context.
Fail-closed	Prisma throws if a tenanted query has no tenantId.
Seed wipe	Running the demo seed resets operational demo data. SEED_IF_EMPTY skips when users exist.
nv-phone	≤639px layout for Employee + IT lookup. Not an admin rewrite.
Keep-alive	GitHub cron ping /api/health so scheduled jobs keep running.
Residency	GET /api/health field. Singapore = demo; India on request. Do not drop the DB to move region.
Prompt N	Internal build increment in PROGRESS.md / DECISIONS.md. Not a public version number.

Appendix A — API surface cheat-sheet

Not exhaustive Swagger. Enough to know where to look. All prefixes sit under /api. Auth: Bearer unless @Public().

Prefix	Module	Notes
--------	--------	-------

/auth	Auth	login, logout, refresh, me, signup, forgot/reset/change password, TOTP
/health	Health	Public. SELECT 1 + region
/tenant	Tenancy	snapshot, branding, onboarding, export, billing, delete
/platform/tenants/:id	Platform	Public + shared secret
/assets	Assets	CRUD + assign/transfer/retire/bulk/audit/qr
/public/assets	PublicAssets	Unauthenticated card + PNG, optional /t/:slug/:code
/employees	Employees	CRUD, offboard, history, /me, checklists
/locations /departments /categories	Org	IT Admin+ writes
/accessories /consumables /issue-kits	Stock	checkout/issue/adjust
/asset-requests	Requests	review + fulfill
/maintenance	Maintenance	Team module
/support-tickets /ticket-* /canned-responses	Helpdesk	Also /email-in/*
/chat	Chat	HTTP + WS namespace /chat
/vendors /purchase-requisitions /purchase-orders /vendor-contracts /procurement	Procurement	Team module
/dashboard /search /reports	Home	setup is admin-only
/import /export /import-jobs /saved-views /reconciliation	Governance	
/notifications /warranty/run-check	Alerts	
/audit-logs	Audit	export CSV; Super Admin/IT Admin
/notes /records	Notes/overrides	
/webhooks /users	Settings	

Appendix B — Related in-repo documents

Read these after this handoff, not instead of it. They drift.

File	Use for
README.md	Quick start commands (verify env names against §6)
DECISIONS.md	Why a constraint exists. Newest notes at the bottom of each phase.
PROGRESS.md	What each Prompt shipped
FUTURE_IDEAS.md	Out of scope. Do not implement from here without a product decision.
PRODUCT_GAPS_AND_ENHANCEMENTS.md	Prompt 32 security/loops ledger
QA_TEST_PLAN.md / QA_RESULTS.md / QA_DEFECTS.md	Last QA sitting (Help + axe; mailbox still open)
docs/BACKUP.md	pg_dump / restore drill
design-reference/DESIGN_TOKENS.md	Colour/type/shell numbers
PROJECT_STATUS.md / PROJECT_HISTORY.md / PROJECT_DOCUMENTATION.md	Historical; corroborate against code

How this handoff was produced

Explored the repo on 16 September 2026: Prisma schema, Nest app.module and every module folder, frontend App.tsx/access.ts/providers, compose, render.yaml, CI, DECISIONS/PROGRESS/QA, and env usage. Diagrams were generated as PNG (matplotlib) and embedded here because Word does not render Mermaid. A parallel markdown handover lives under docs/ (DEVELOPER_HANDOVER.md, ARCHITECTURE.md, API_REFERENCE.md, FUNCTIONAL_SPECIFICATION.md, etc.). Items the code could not prove are marked Unclear or Assumed in §§2, 6, and 10.

Markdown handover (docs/)	Use for
DEVELOPER_HANDOVER.md	Index + scorecard; links to split files
ARCHITECTURE.md	Mermaid/ASCII diagrams (export separately if needed)
API_REFERENCE.md	Integrations; full route list in handoff/api-routes-index.md
FUNCTIONAL_SPECIFICATION.md	Behavior, lifecycles, QA-oriented rules
SETUP_AND_DEPLOYMENT.md	Phase 11 verification log

If you can ask only one person one question after reading this, ask whether production still allows demo logins and whether Resend/IMAP secrets have been pasted — those two facts are operator state, not git state.